

From Vibe Coding to Spec-Driven Agentic Engineering

A Hands-On Implementation using Claude Code

PART V

Instructor: Muhammad Arif Butt, PhD

About The Instructor



Dr. Muhammad Arif Butt
Asst. Prof. Department of Data Science
University of Punjab, Lahore
arif@pucit.edu.pk
https://arif.excaliat@gmail.com
https://arifbutt.me
https://www.linkedin.com/in/dr-arif-butt/
https://youtube.com/learnwitharif
https://github.com/arifpucit
https://huggingface.co/arif-butt

- **Education:**
  - Graduation from Pakistan Military Academy, Kakul
  - MPhil CS from University of Punjab, Lahore
  - PhD CS from University of Punjab, Lahore
- **Experience:**
  - Served in field/staff/instructional posts in Pakistan Army
  - Assistant Professor, Department of Data Science
  - Founder Excaliat (https://excaliat.com/en)
  - Founder FalconHunt (https://falconhunt.org/)
  - Co-Founder Tbox Solutionz (https://tboxsolutionz.com/)
- **Research Interest:**
  - Embedded and Real-Time Operating Systems
  - Vulnerability Analysis, Binary Exploitation & Exploit Development
  - AI-Driven Cybersecurity and Securing AI systems

Scheme of Sessions

Part - I: Overview of Agentic Coding Assistants

Part – II: Claude Ecosystem, Installing & using Claude Code

Part – III: Slash Commands, Context Window & `.claude` Directory

Part – IV: Workflow of Building a Basic Vulnerable Web Application

Part – V: Exploiting Vulnerable Web Application

Part – VI: Workflow of Implementing Features and Fixing Bugs

Part – VII: Skill Marketplace and Writing your Own Skills

Part – VIII: Creating and using Subagents & Agent Teams

Part – IX: Model Context Protocol

Part – X: Creating and using Hooks

Part – XI: Creating and using Plugins

Part 5
Exploiting Vulnerable Web App

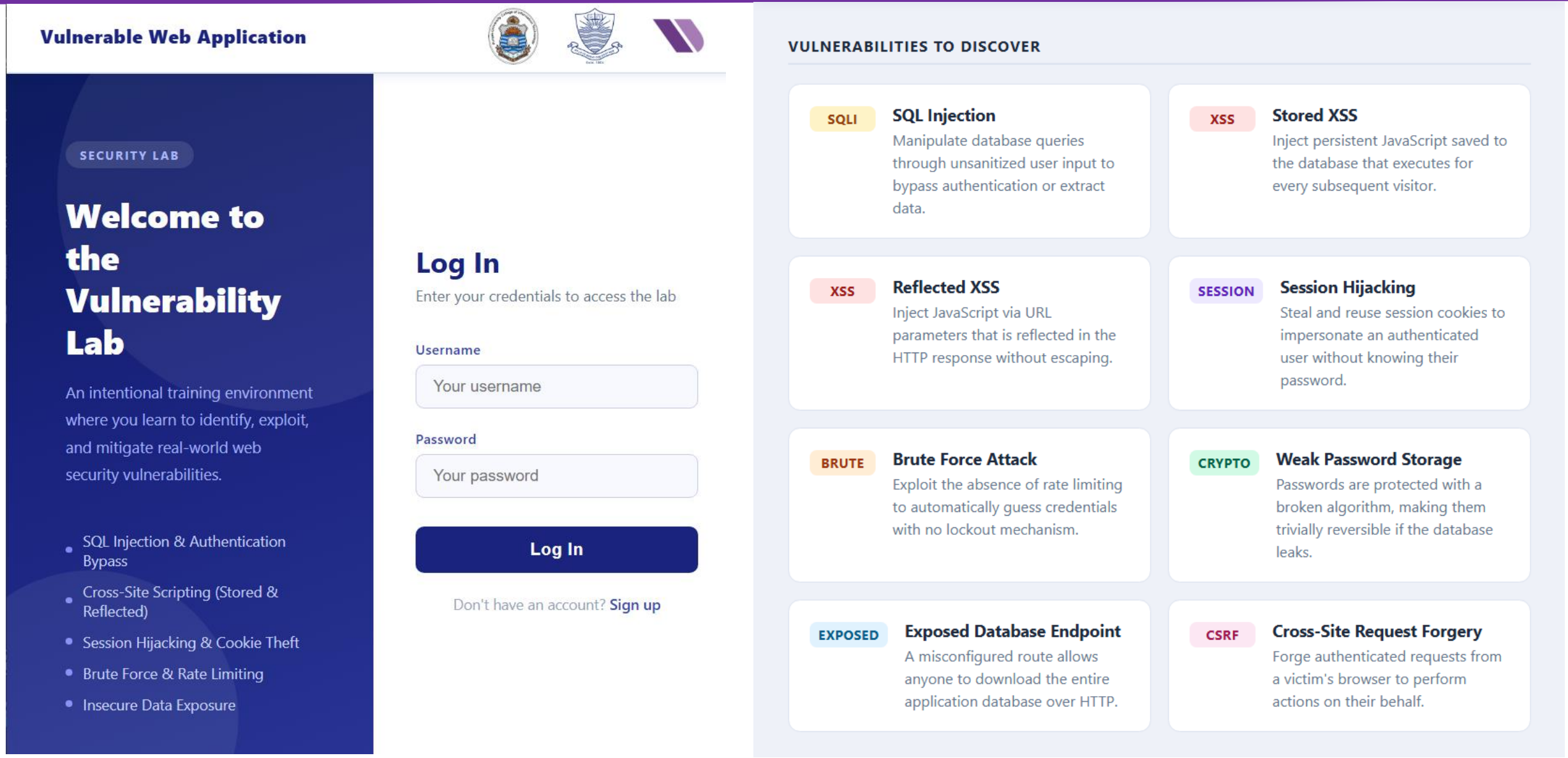
Instructor: Muhammad Arif Butt, PhD

OWASP Top-10 Web App Vulnerabilities

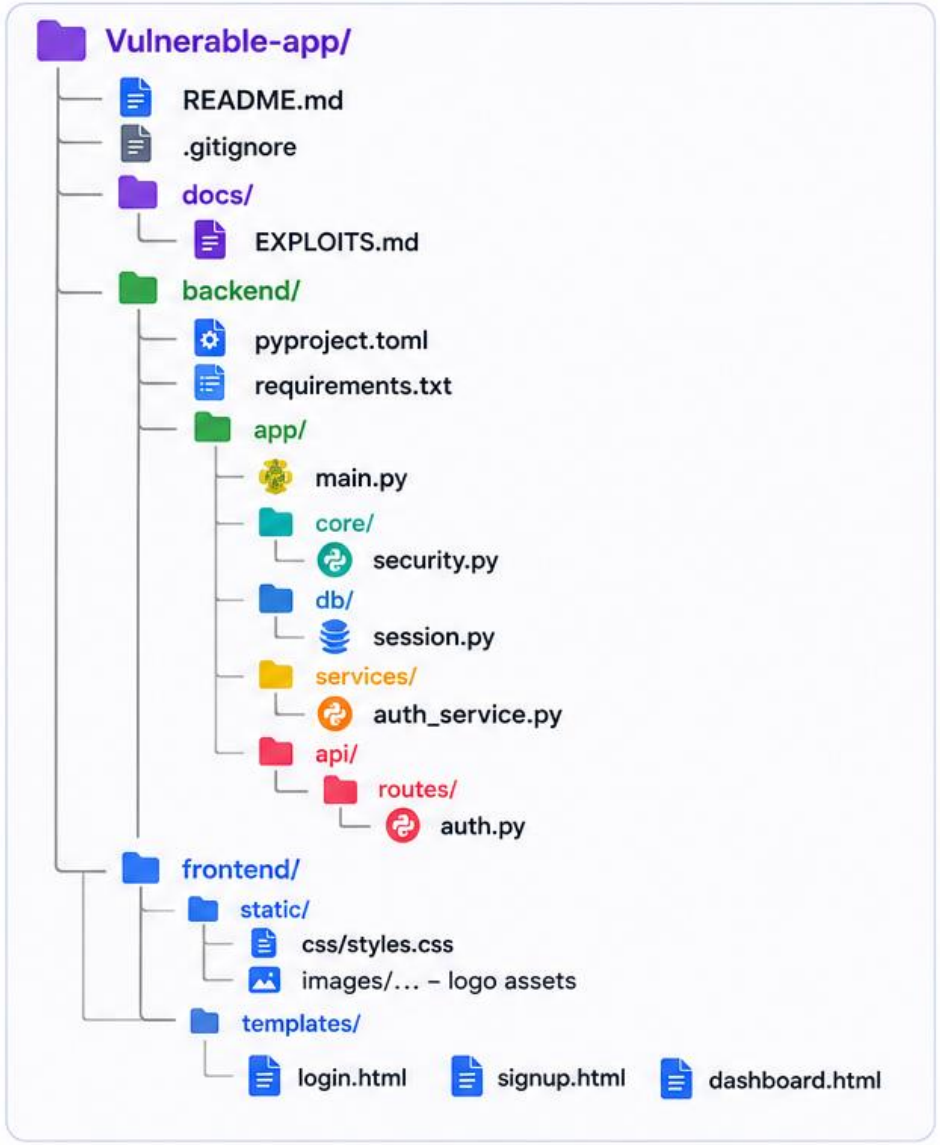
2013	2017	2021	2025
A1 Injection	A1 Injection	A1 Broken Access Control	A1 Broken Access Control
A2 Broken Authentication and Session Management	A2 Broken Authentication	A2 Cryptographic Failures	A2 Security Misconfiguration
A3 Cross-Site Scripting (XSS)	A3 Sensitive Data Exposure	A3 Injection	A3 Software Supply Chain Failures
A4 Insecure Direct Object References	A4 XML External Entities (XXE)	A4 Insecure Design	A4 Cryptographic Failures
A5 Security Misconfiguration	A5 Broken Access Control	A5 Security Misconfiguration	A5 Injection
A6 Sensitive Data Exposure	A6 Security Misconfiguration	A6 Vulnerable and Outdated Components	A6 Insecure Design
A7 Missing Function Level Access Control	A7 Cross-Site Scripting (XSS)	A7 Identification and Authentication Failures	A7 Authentication Failures
A8 Cross-Site Request Forgery (CSRF)	A8 Insecure Deserialization	A8 Software and Data Integrity Failures	A8 Software or Data Integrity Failures
A9 Using Components with Known Vulnerabilities	A9 Using Components with Known Vulnerabilities	A9 Security Logging and Monitoring Failures	A9 Security Logging and Alerting Failures
A10 Unvalidated Redirects and Forwards	A10 Insufficient Logging & Monitoring	A10 Server-Side Request Forgery (SSRF)	A10 Mishandling of Exceptional Conditions

https://owasp.org/www-project-top-ten/

Vulnerable Web App Interface



Directory Structure



README.md – overview, install/run instructions, list of intentional bugs

docs/EXPLOITS.md – step-by-step exploitation guide for every flaw

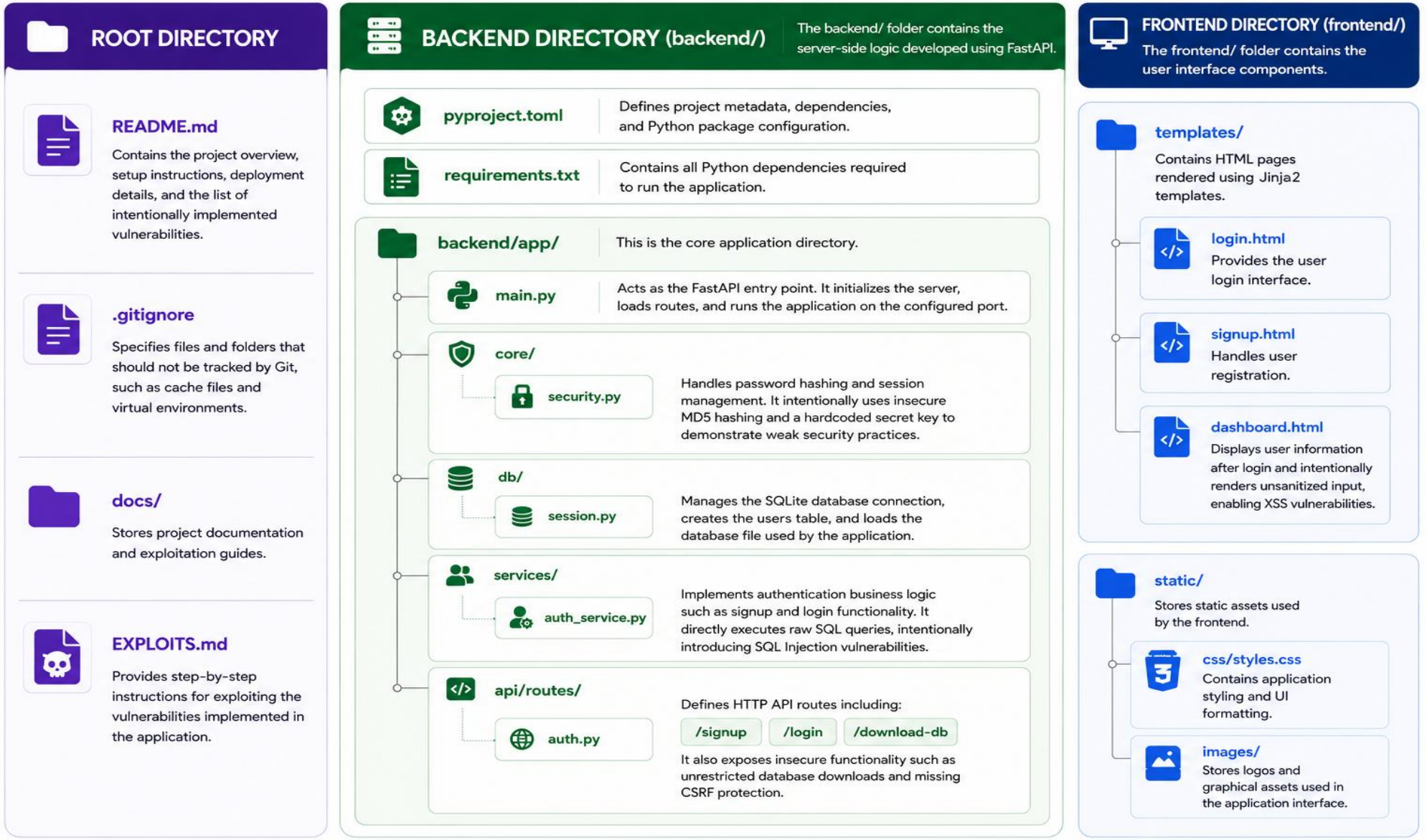
backend/ – FastAPI backend (Python)

- **main.py** – FastAPI entry point; creates the server and mounts routes
- **core/security.py** – password handling (MD5 hash) and session-secret constant
- **db/session.py** – SQLite connection, creates users table, loads the DB file
- **services/auth_service.py** – business logic for signup / login, queries DB directly
- **api/routes/auth.py** – HTTP endpoints: /signup, /login, /download-db

frontend/ – Web UI

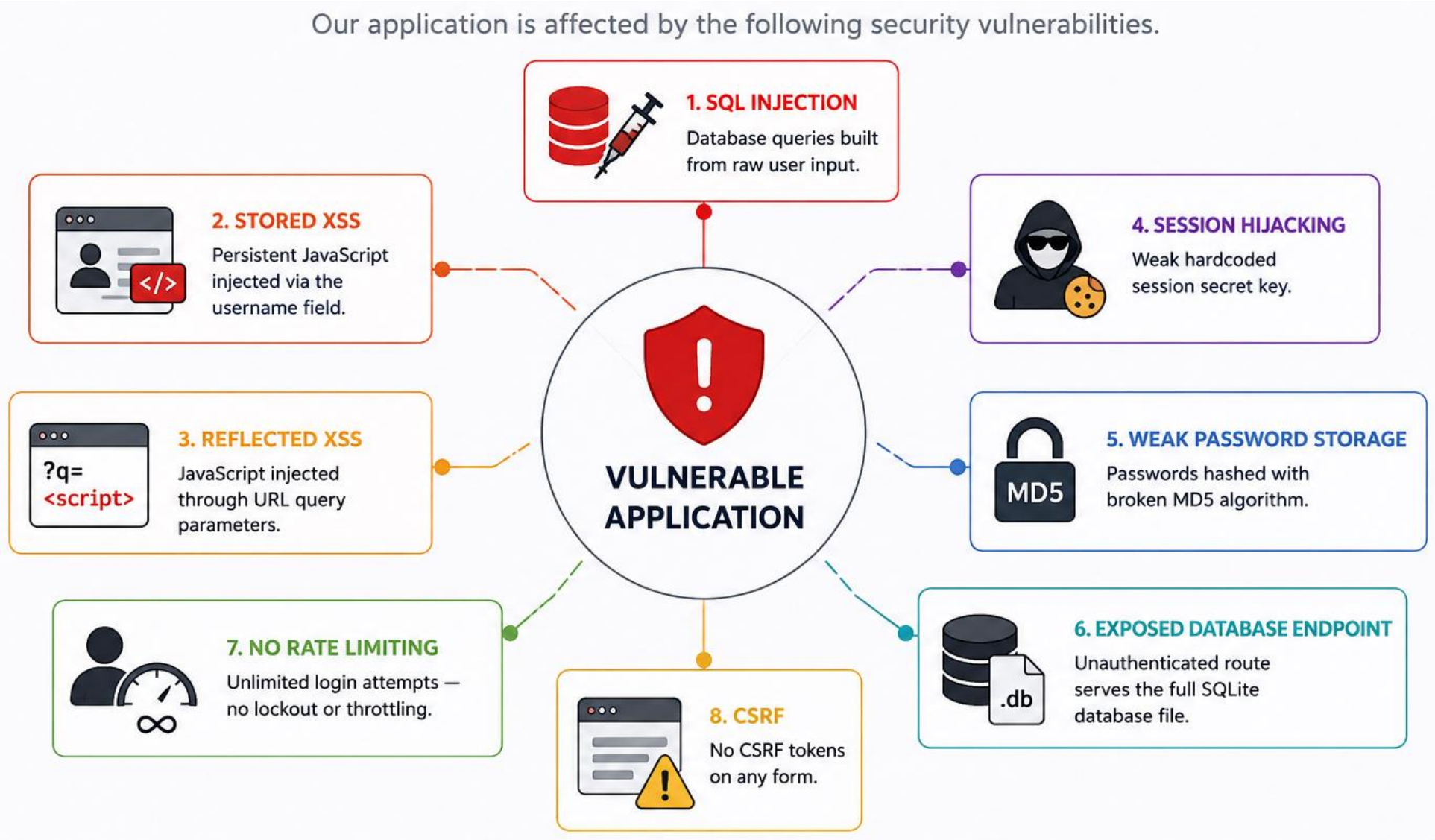
- **static/** – CSS and images (logo assets, etc.)
- **templates/** – HTML pages for the application (login, signup, dashboard)

Directory Structure (cont...)

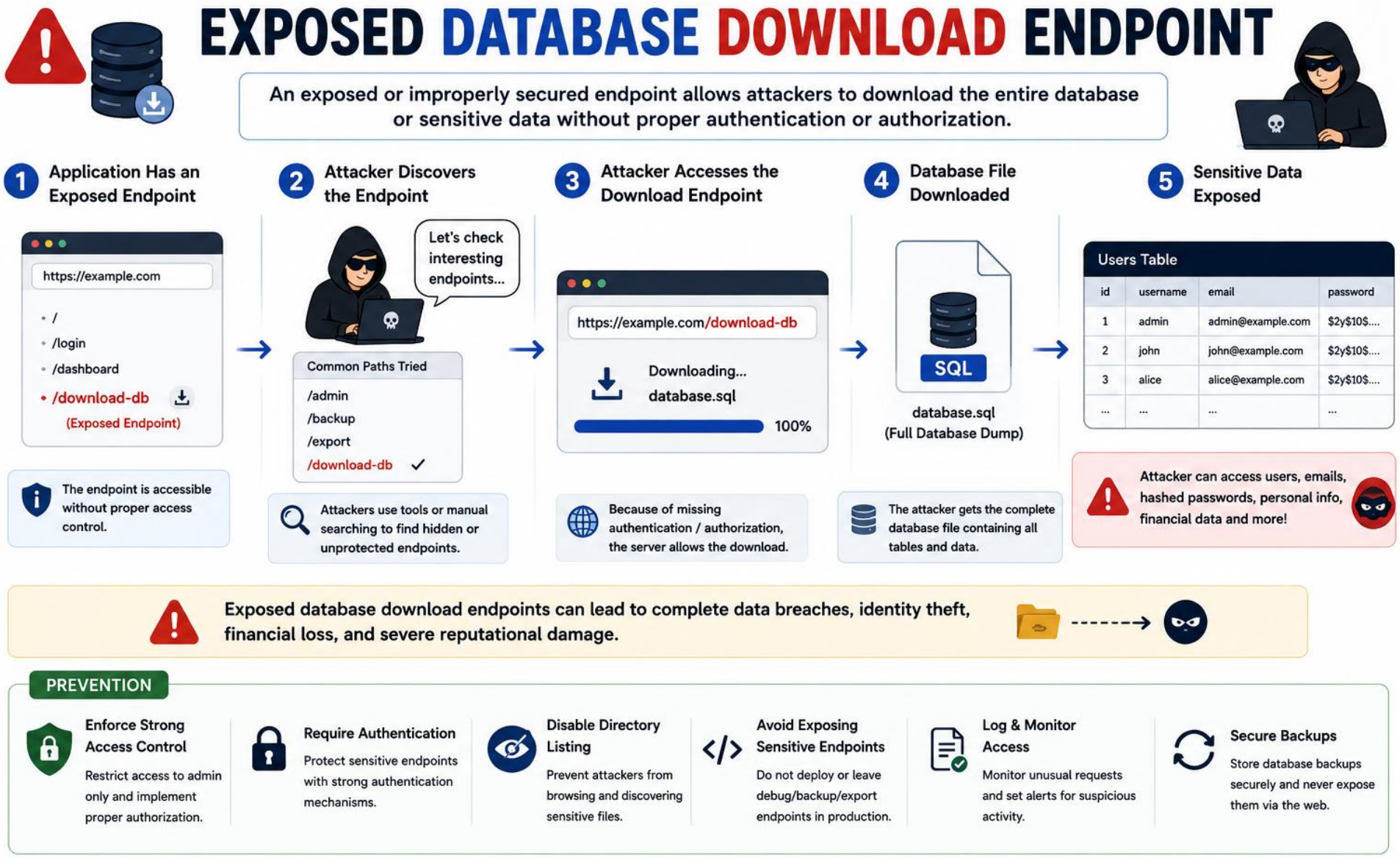


https://github.com/arifpucit/vuln-web-app

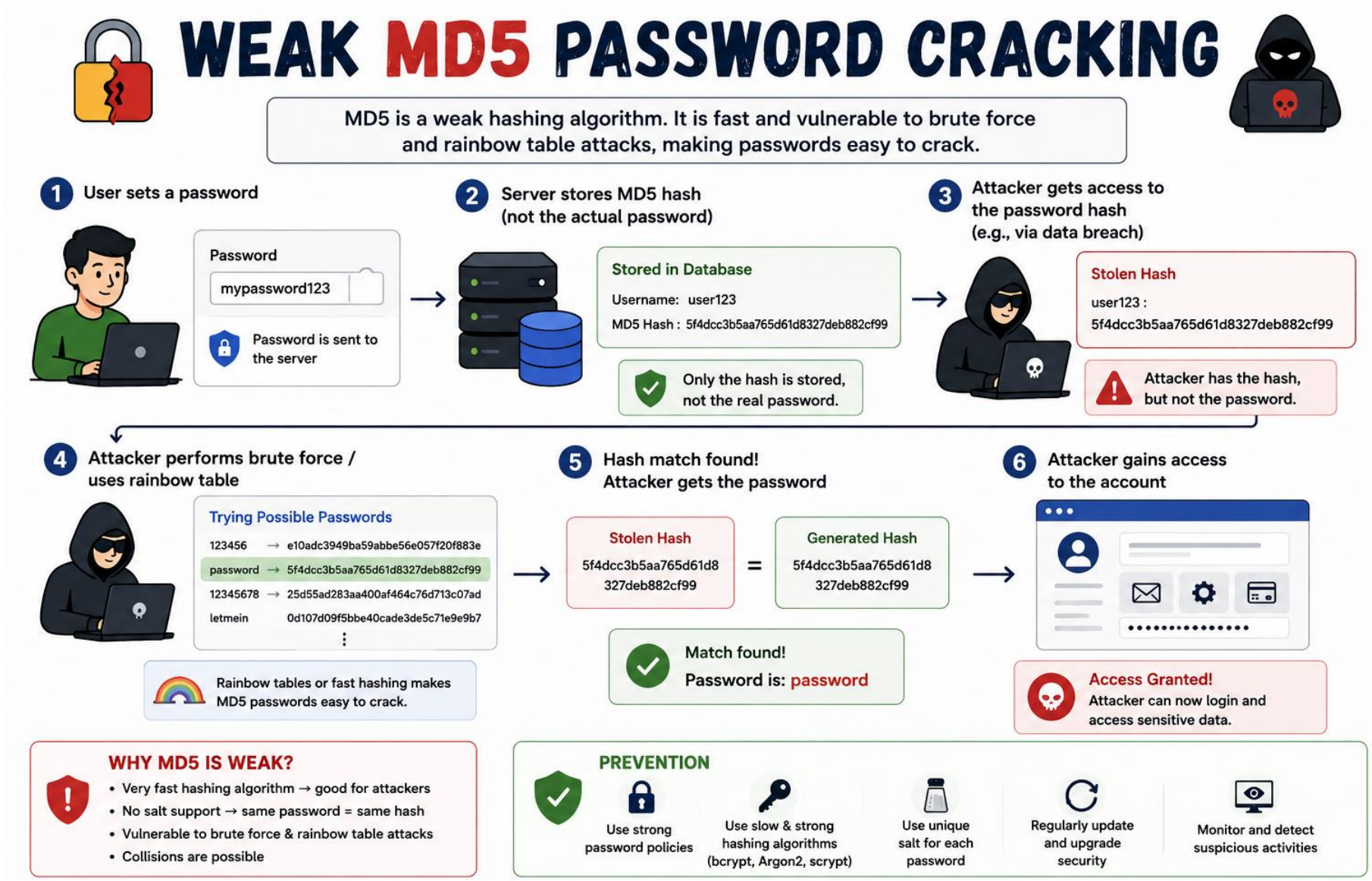
Vulnerabilities



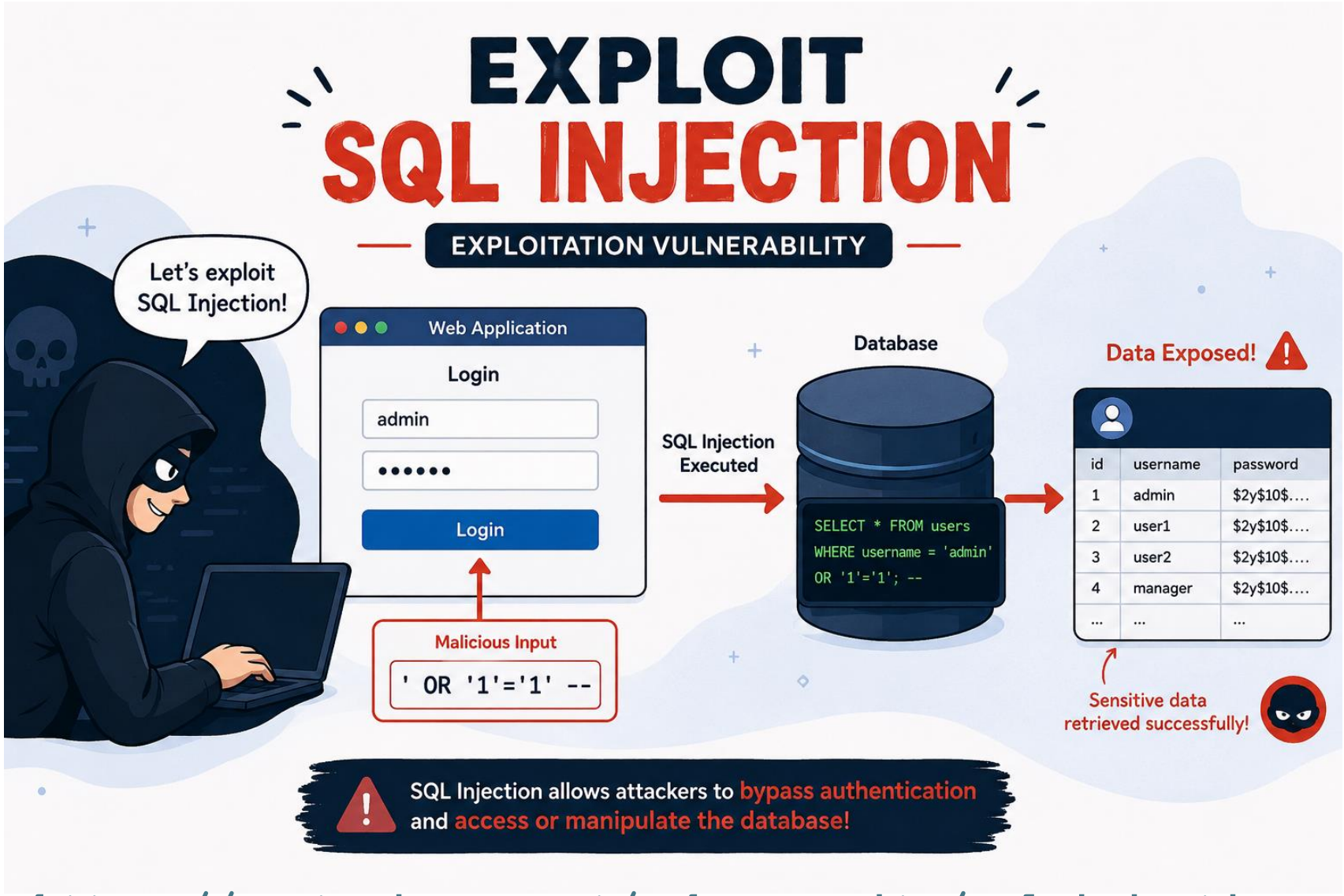
Exposed Endpoint: Attack Vector



Weak Password Encryption: Attack Vector

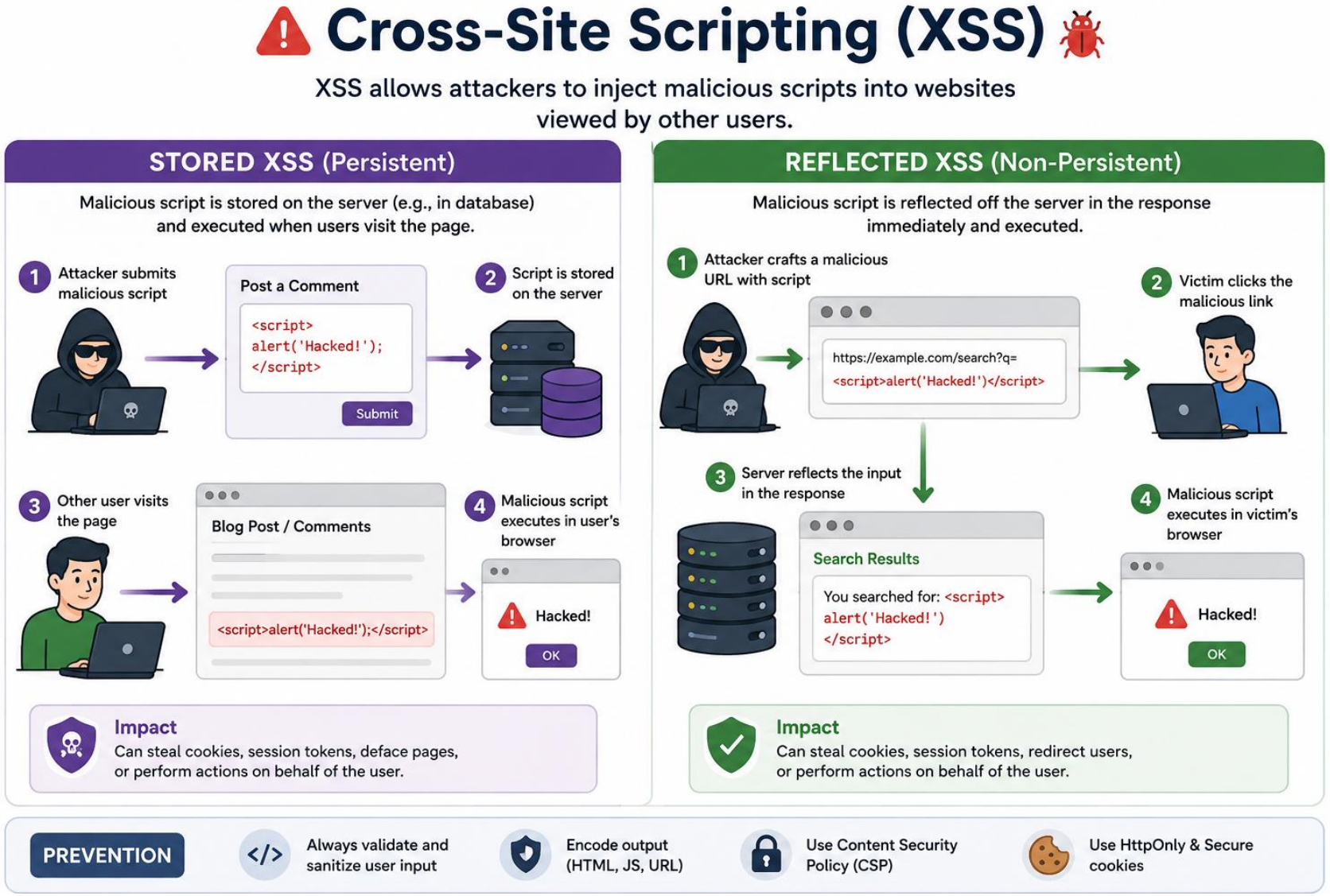


SQLi: Attack Vector



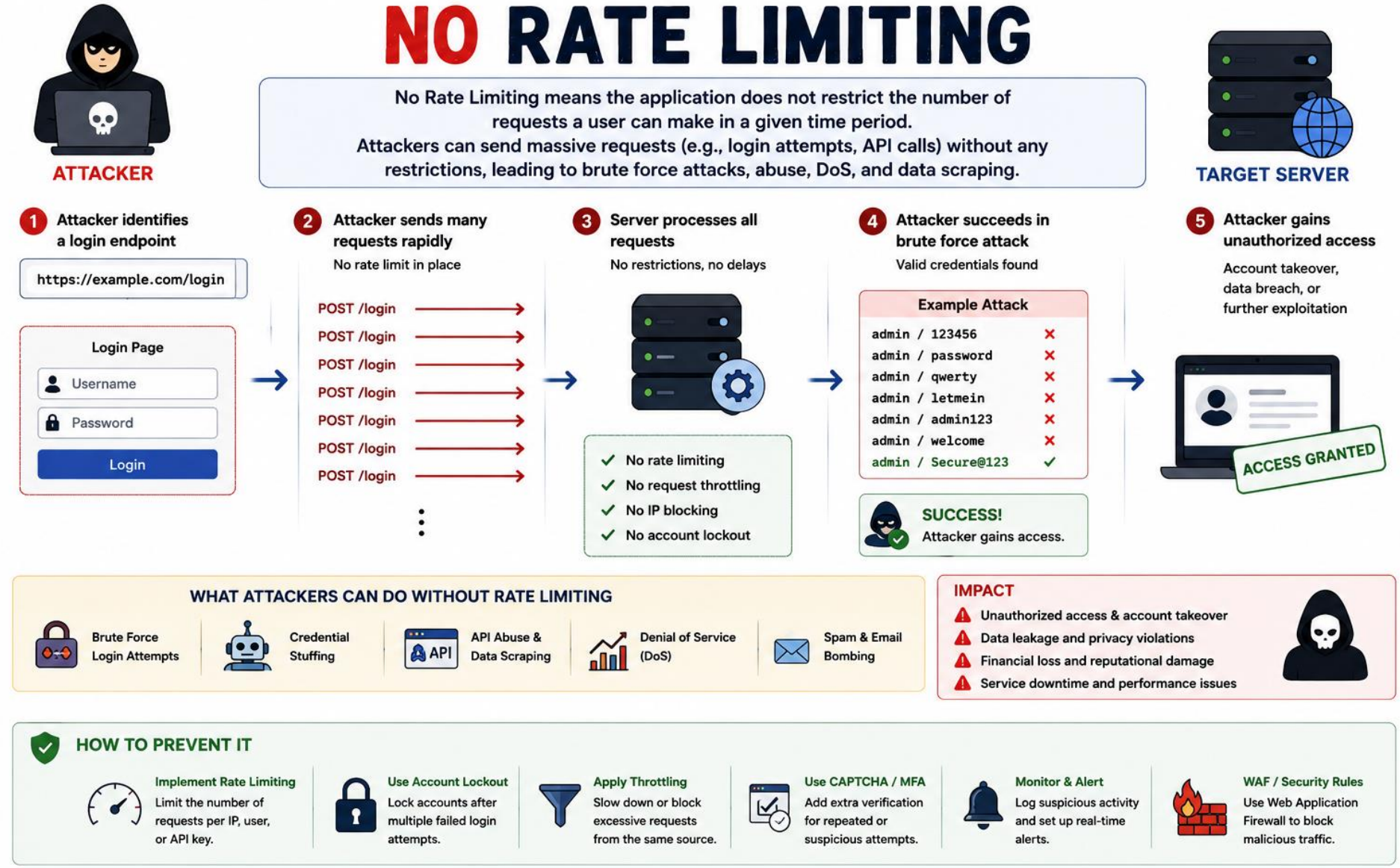
https://portswigger.net/web-security/sql-injection

XSS: Attack Vector

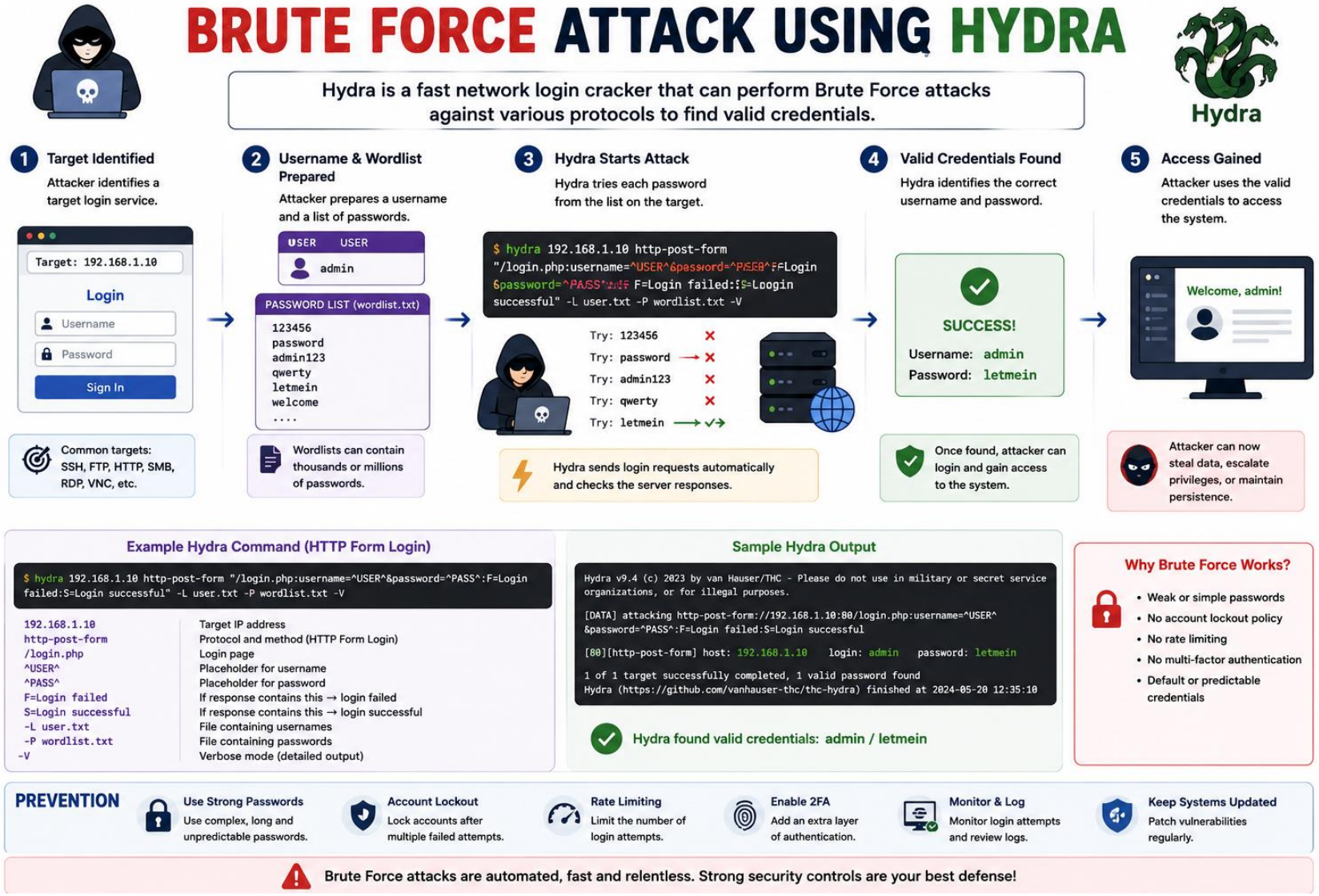


https://portswigger.net/web-security/cross-site-scripting

No Rate Limiting: Attack Vector



Brute Force: Attack Vector



https://portswigger.net/web-security/authentication

Session Management

1 INITIALIZATION

You log in successfully, so the server creates a unique **Session ID** and sends it to your browser inside a cookie.

2 CONTINUATION

Your browser auto-sends that ID with **every request**. The server reads it to recognize you and load your data.

3 TERMINATION

The session ends when you **log out** or after an inactivity timeout — the Session ID then stops working.

Anatomy of the `set-cookie` header

the whole line is the COOKIE
the highlighted value is the SESSION ID

YOU LOG IN — the request you send:

```
% curl -i -X POST http://127.0.0.1:3001/login -d "username=arif&password=kakamanna"
```

↓ the server replies with this header:

```
set-cookie: session=eyJ1c2Vy…b20ifQ==.aiatFA.EsXvnP…IuU0s ; path=/; Max-Age=1209600;
httponly; samesite=lax
```

```
base64( payload ) → {"user_id": 1, "username": "arif", "email": "arif@gmail.com"}
```

base64 is encoding, not encryption — anyone holding the cookie can read this; the signature only blocks tampering.

WHAT EACH PART MEANS

Cookie name

`session` — the label the browser files this cookie under and sends back on each request.

Session ID — the value it carries

What actually identifies you. Three parts: `payload . timestamp . signature` (shortened on the left). The payload is base64 — decoded below.

Cookie attributes

Rules, not identity: `Max-Age` = 14 days · `httponly` (JavaScript can't read it) · `samesite=lax`.

Types of Cookies

Persistent has Max-Age / Expires

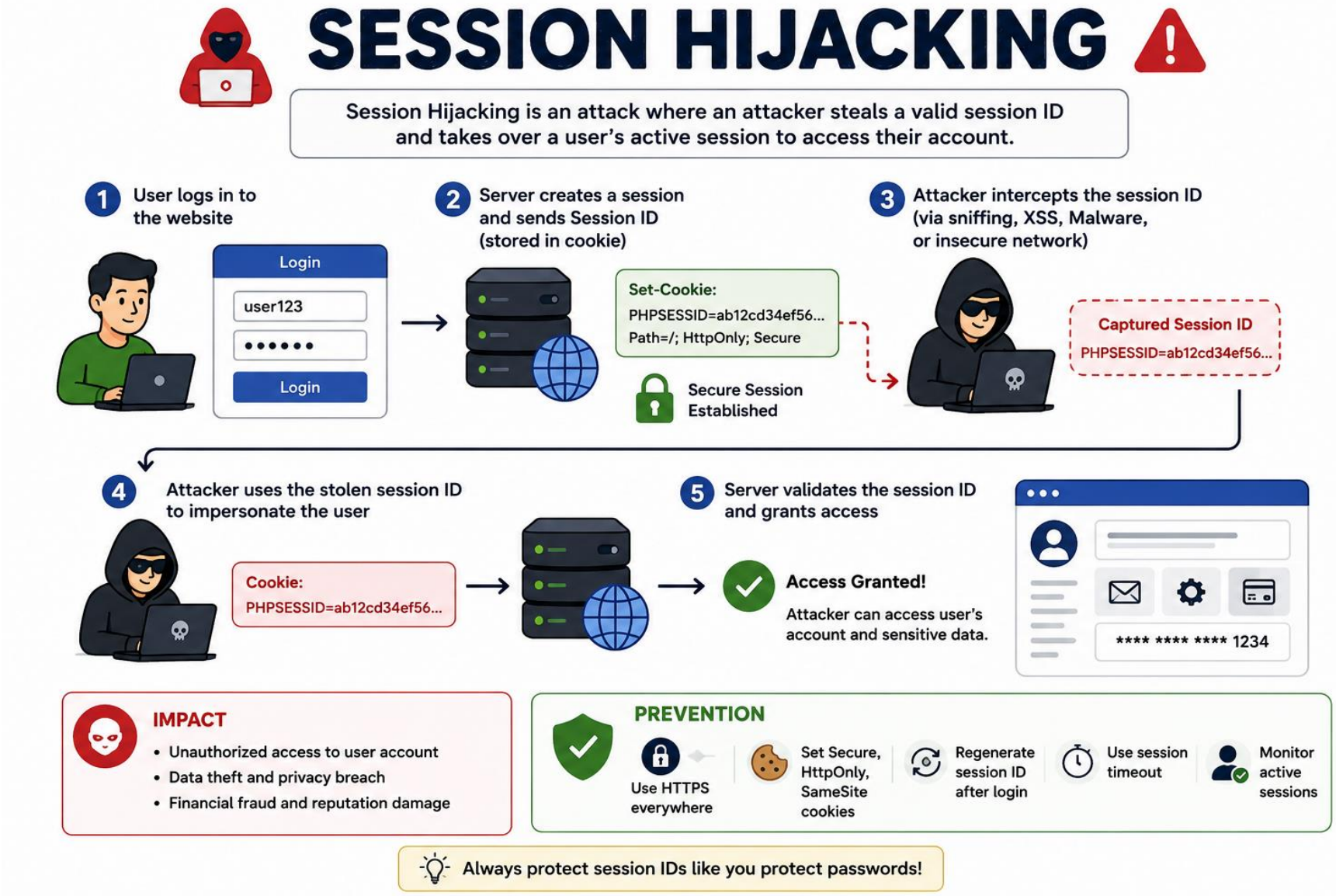
Saved on disk; survives a browser restart and lasts until its expiry — like the 14-day cookie above.

Session no Max-Age / Expires

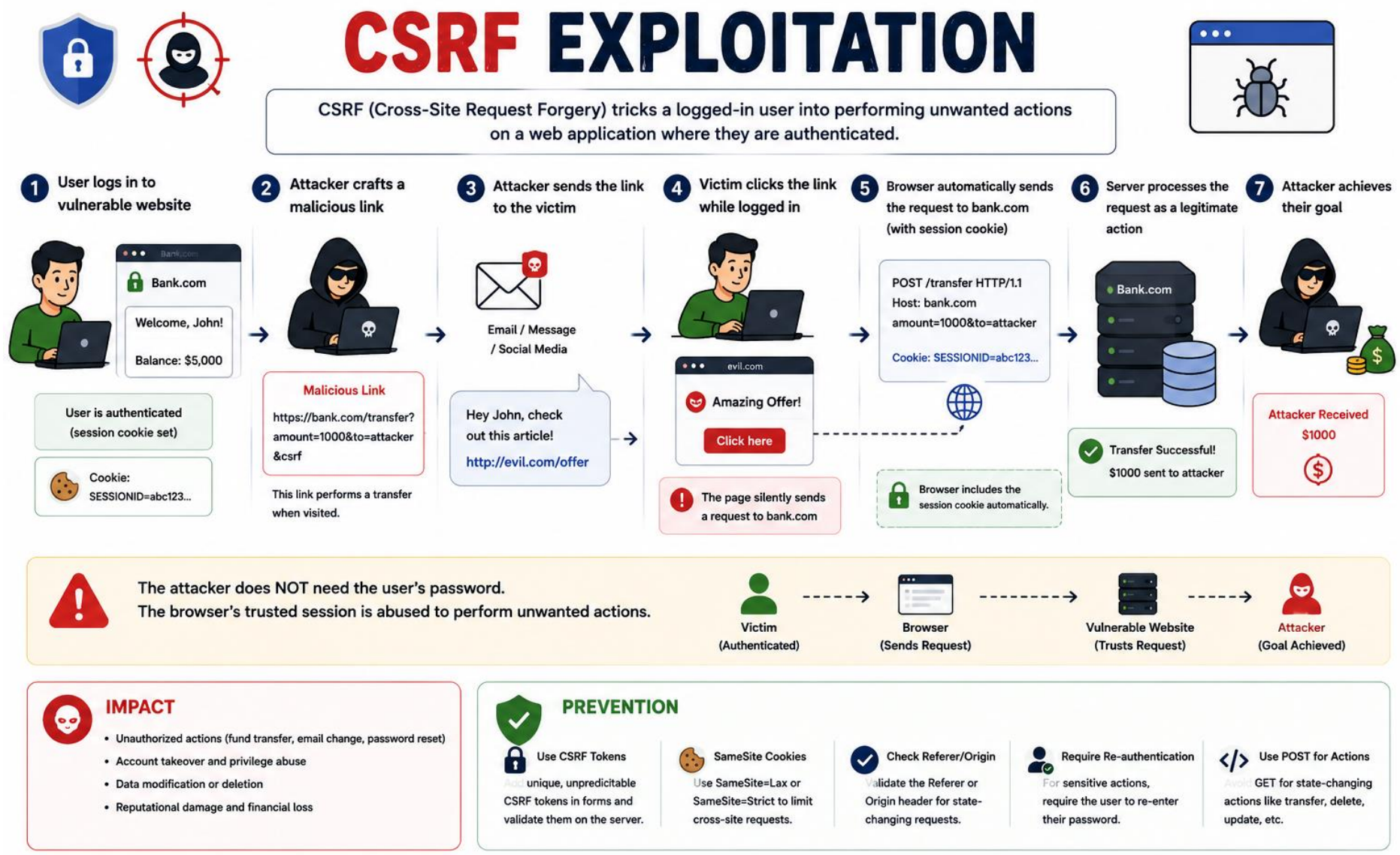
Kept only in the browser's memory; deleted the moment you close the browser.

The SID is the membership number, while the cookie is the card that carries it.

Session Hijacking: Attack Vector



CSRF: Attack Vector



https://portswigger.net/web-security/csrf

The OWASP Top-10 Family

01 — FOUNDATIONAL APPLICATION SECURITY

2003 – 2025

A01-A10

OWASP TOP 10

Web Applications

The flagship list. Broken access control, injection, cryptographic & supply-chain failures in web apps.

INTRODUCED	LATEST
2003	2025

M1-M10

OWASP MOBILE TOP 10

Mobile Apps

Risks for iOS / Android apps: improper credential use, insecure storage, weak auth, supply chain.

INTRODUCED	LATEST
2014	2024

API1-API10

OWASP API SECURITY TOP 10

APIs

API-specific risks: broken object-level authorization (BOLA), broken auth, unsafe consumption.

INTRODUCED	LATEST
2019	2023

02 — THE AI-NATIVE ERA

2023 – 2026

LLM01-10

OWASP TOP 10 FOR LLMS

LLM Applications

Generative-AI risks: prompt injection, sensitive-info disclosure, data / model poisoning, excessive agency.

INTRODUCED	LATEST
2023	2025

MCP01-10

OWASP MCP TOP 10

Model Context Protocol

Securing the agent–tool protocol: tool poisoning, token leakage, shadow servers, excessive scope.

INTRODUCED	LATEST
2025	2025 β

ASI01-10

OWASP TOP 10 FOR AGENTIC APPS

Autonomous Agents

Acting AI systems: goal hijacking, tool misuse, identity / delegation gaps, rogue & cascading agents.

INTRODUCED	LATEST
2026	2026

AST01-10

OWASP AGENTIC SKILLS TOP 10

Agent Skills

The agent "behavior layer": malicious & over-privileged skills, supply-chain compromise, the lethal trifecta.

INTRODUCED	LATEST
2026	2026 β

Q/A



Thank you!